UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

IF 451
Advance Web Programming

Meet12 - File Upload & Basic Authentication

Wirawan Istiono, S.Kom., M.Kom

Meet 11

Objective:

Sub-CLO0713 - Students are able to implement various types of libraries and database to create web application using Node JS (C3)

Sub-CLO0714 - Students are able to implement Node JS Framework to build Web application (C3, C6)

OUTLINE

- *File upload using multer.*
- *Login with username/password stored in MySQL Database*

File upload using multer.

What is Multer?

- Multer is a middleware for ExpressJS that handles file uploads.
- It works with the multipart/form-data format, which is the standard way browsers send files (for example, when you upload an image via a form).
- Without Multer, Express cannot directly read uploaded files because multipart/form-data is more complex than JSON or URL-encoded data.

How Multer Works

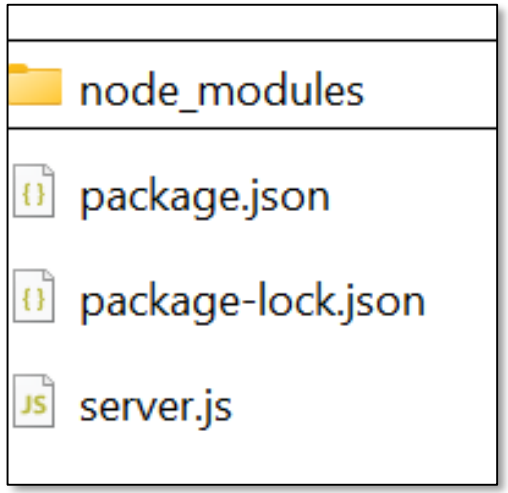
- You create an upload middleware using Multer.
- You tell Multer where to store the file (e.g., in the local uploads/ folder).
- In your Express route, you use the Multer middleware to handle the file(s).
- Multer will make the file information available on req.file (for single upload) or req.files (for multiple uploads)

Install Multer

Write the command below in your cmd to install multer (and express)

npm install express

npm install multer



Create HTML files and Folder Structure

To create HTML input form to upload file, create structure folder like the sample below:

```
project/
│
├── server.js
├── uploads/
└── public/
      └── index.html
```

Sample HTML code in index.html:

```
<body>
  <h2>Upload a File</h2>
  <form action="http://localhost:3000/upload" method="post" enctype="multipart/form-data">
    <input type="file" name="myfile" />
    <button type="submit">Upload</button>
  </form>
</body>
```

Using Multer to call index.html and upload file

Write the sample code below in server.js file

```
const express = require("express");
const multer = require("multer");
const path = require("path");

const app = express();

//call index.html
app.use(express.static("public"));
```

```
// Multer storage setup
const storage = multer.diskStorage({
    destination: function (req, file, cb) {
        cb(null, "uploads/");
    },
    filename: function (req, file, cb) {
        cb(null, Date.now() + path.extname(file.originalname));
    }
});

const upload = multer({ storage: storage });

app.post("/upload", upload.single("txtFile"), (req, res) => {
    console.log(req.file);
    res.send("File uploaded successfully!");
});

app.listen(3000, () => {
    console.log("Server running at http://localhost:3000");
});
```

Result Upload file with Multer

Run server.js with command in cmd:

node server.js


```
{
  fieldname: 'txtFile',
  originalname: 'Picture1.jpg',
  encoding: '7bit',
  mimetype: 'image/jpeg',
  destination: 'uploads/',
  filename: '1759475668433.jpg',
  path: 'uploads\\1759475668433.jpg',
  size: 50206
}
```

Multiple File Upload

Try to changing the code in server.js with the sample code below to multiple file upload

```
<body>
    <h2>Upload a File</h2>
    <form action="http://localhost:3000/upload" method="post" enctype="multipart/form-data">
    <input type="file" name="txtFile" multiple />
    <input type="submit" value="Upload" >
    </form>
</body>
```

```
/*
app.post("/upload", upload.single("txtFile"), (req, res) => {
    console.log(req.file);
    res.send("File uploaded successfully!");
});
*/

app.post("/upload", upload.array("txtFile", 5), (req, res) => {
    console.log(req.files);
    res.send("Multiple files uploaded!");
});
```

Login with username/password stored in MySQL Database

Preparing Table users

Create table user in your MySQL database using phpMyAdmin or CLI, with detail like the sample below.

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL
);
```

Frontend: Create Signup html file

Create new file with name **signup.html** in public folder and write the sample code below

```
<h2>Signup</h2>
<form action="/signup_proses" method="POST">
    Username: <input type="text" name="txtUsername" required><br>
    Password: <input type="password" name="txtPassword" required><br>
    Retype Password<input type="password" name="txtRetype" required><br>
    <Input type="submit" value="signup" >
</form>
```

Install Depedency

Don’t forget to install Depedencies

```
npm init -y
npm install express mysql2 body-parser
```

Don’t forget to Turnon your mysql and Apache (if needed) in XAMPP

Backend: Create koneksi.js to connect with mysql

```
const mysql = require("mysql2");

const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database:“yourDatabaseName"
});

db.connect(err => {
  if (err) throw err;
  console.log("Connected to MySQL");
});

module.exports = db;
```

Edit server.js to call signup.html

Add or edit the code below in server.js file

```
app.use(express.urlencoded({ extended: true }));
app.use(express.json());

//call index.html
app.use(express.static("public"));

// Route untuk panggil form signup
app.get("/signup", (req, res) => {
  res.sendFile(path.join(__dirname, "public", "signup.html"));
});

// Route untuk kirim data post signup
const signupProses = require("./signup_proses");
app.use("/", signupProses);
```

Backend: Try to get post variable in signup_proses.js

```
const express = require("express");
const router = express.Router();
const kon = require("./koneksi");

router.post("/signup_proses", (req, res) => {
  const { txtUsername, txtPassword, txtRetype } = req.body;

  if (txtPassword !== txtRetype) {
    return res.send("Password dan retype tidak sama!");
  }

  // Di sini bisa nanti hapus dan ganti query insert ke database MySQL
  res.send(`
    <h3>Signup Success</h3>
    <p>Username: ${txtUsername}</p>
    <p>Password: ${txtPassword}</p>
  `);
});

module.exports = router;
```

Backend: Create signup_proses.js file (Edit)

```
    if (txtPassword !== txtRetype) {
         return res.send("Password dan retype tidak sama!");
    }

    // Query insert ke MySQL
    const sql = "INSERT INTO users (username, password) VALUES (?, ?)";

    kon.query(sql, [txtUsername, txtPassword], (err, result) => {
        if (err) {
            if (err.code === "ER_DUP_ENTRY") {
                return res.send("Username sudah terdaftar!");
            }
            console.error(err);
            return res.status(500).send("Database error");
        }

        res.send(`<h3>Signup Success</h3>`);
    });
```

Frontend: Create Login html file

Create new file with name **login.html** in public folder and write the sample code below

```
<h2>Login</h2>
<form action="/login_proses" method="POST">
    Username: <input type="text" name="txtUsername" required><br>
    Password: <input type="password" name="txtPassword" required><br>
    <Input type="submit" value="Login" >
</form>
```

Add Code Server.js

Add code below in Server.js to create rooting to login_proses.js

```
// Route untuk panggil form signup
app.get("/login", (req, res) => {
  res.sendFile(path.join(__dirname, "public", "login.html"));
});

// Route untuk kirim data post login
const loginProses = require("./login_proses");
app.use("/", loginProses);
```

Add login_proses.js for get POST data

Add code below in login_proses.js

```
const express = require("express");
const router = express.Router();
const kon = require("./koneksi");

router.post("/login_proses", (req, res) => {
    const { txtUsername, txtPassword } = req.body;

    //code select to check username and password here
    res.send(txtUsername + " / " + txtPassword);
});
module.exports = router;
```

Add login_proses.js for get POST data (Edit)

```
    // Query untuk cek username dan password
    const sql = "SELECT * FROM users WHERE username = ? AND password = ?";
    kon.query(sql, [txtUsername, txtPassword], (err, result) => {
        if (err) {
            console.error("Error saat query:", err);
            return res.status(500).send("Terjadi error pada server.");
        }

        if (result.length > 0) {
            res.send("Login Sukses, welcome " + txtUsername);
        } else {
            res.send("Login Gagal");
        }
    });
```

REFERENCES

- B. Lawson and J. Encinas, Node.js Web Development, 6th ed., Packt Publishing, 2023
- R. Raj, Learning Node.js Development, Packt Publishing, 2018.
- E. Cantelon, M. Harter, T. Holowaychuk, and N. Rajlich, Node.js in Action, 2nd ed., Manning Publications, 2017.
- Maulana, a., bau, r.t.r., munawar, z., setiawan, r., aisa, s., istiono, w., irfan, a., pratama, y.a., ramadhany, e.d., pomalingo, s. And permana, a.a., 2023. *Pemrograman web 101: memahami dasar-dasar untuk mengembangkan situs web*. Get press indonesia.
- The Node.js Foundation, "Node.js Documentation," [Online]. Available: https://nodejs.org/en/docs/. [Accessed: Jun. 5, 2025].
- Express.js Team, "Express.js Documentation," [Online]. Available: https://expressjs.com/. [Accessed: Jun. 5, 2025].

Visi

Menjadi Program Studi Strata Satu Informatika **unggulan** yang menghasilkan lulusan **berwawasan internasional** yang **kompeten** di bidang Ilmu Komputer (*Computer Science*), **berjiwa wirausaha** dan **berbudi pekerti luhur**.

INFORMATIKA
UMN
UNIVERSITAS MULTIMEDIA NUSANTARA

Misi

1. Menyelenggarakan pembelajaran dengan teknologi dan kurikulum terbaik serta didukung tenaga pengajar profesional.
2. Melaksanakan kegiatan penelitian di bidang Informatika untuk memajukan ilmu dan teknologi Informatika.
3. Melaksanakan kegiatan pengabdian kepada masyarakat berbasis ilmu dan teknologi Informatika dalam rangka mengamalkan ilmu dan teknologi Informatika.